



PUSL3190 Computing Project

Project Initiation Document (PID)

Local Browser Isolation With Download File
Activity Monitoring

Supervisor: Mr. Chamara Dissanayake

Name: Kotigampala Amaranath

Plymouth Index Number: 10953251

Degree Program: BSc (Hons) Computer Security

Table of Contents

Chapter 01: Introduction	4
Background and Context	4
Problem Statement	5
Scope and Limitations	5
Expected Impact and Stakeholders	6
Chapter 02: Business Case	7
2.1 Business Need	7
2.2 Business Objectives	8
Objective 1: Cost Reduction	8
Objective 2: Security Effectiveness	8
Objective 3: Performance Maintenance	8
Objective 4: Operational Efficiency	8
Chapter 03: Project Objectives	9
Objective 1: Implement Firecracker-based Browser Isolation	9
Objective 2: Develop Real-time File Monitoring System	9
Objective 3: Build User-Friendly Browser Interface	9
Objective 4: Implement Behavioral Analysis Engine	9
Chapter 04: Method of Approach	10
System Architecture Components	10
Data Collection and Security Testing	10
Detection and Analysis Techniques	11
Performance Optimization Strategy	11
Development Tools and Technologies	12
Chapter 05: Conceptual Diagram	13
Diagram Explanation:	13
Chapter 06: Initial Project Plan	15
Timeline and Milestones	15
Resource Requirements	15
Hardware Resources:	15
Software Resources:	15
Knowledge Resources:	15
Chapter 07: Risk Analysis	16
Risk 1: Firecracker Compatibility Issues	16
Risk 2: Browser Integration Complexity	16

Risk 3: Performance Targets Unmet	16
Risk 4: Detection Accuracy Issues	17
Risk 5: Limited Malware Sample Access	17
References	18

Chapter 01: Introduction

Background and Context

Web browsers have become the primary attack vector for cyber threats, with browser-based phishing attacks increasing by 198% in the second half of 2023 compared to the first half of year (Menlo Security, 2024). According to recent cybersecurity statistics, browser exploits ranked second among all attack methods, for nearly 12% of cyberattacks globally between 2021 and 2023 (Keepnet, 2025).

The cybersecurity landscape shows that 70% of users are being affected by malware through their browsers effect into organizations Itself, and 48% of organizations experienced information theft via browser-delivered malware (Terranova Security, 2024).

Traditional security measures such as antivirus software and secure web gateways may detect the attack but most of the time they struggle to detect the attacks, with over 170,000 Data breach and browse exploration identified in 2024, reflecting a 130% increase from 2023 (Infosecurity Magazine, 2025).

Current browser security approaches fall into two categories:

1. Remote Browser Isolation (RBI)
2. Local endpoint protection

Each with significant limitations that create security gaps in enterprise environments. Remote Browser Isolation has emerged as an industry solution where browsing activity occurs on cloud-based servers, transmitting only safe visual representations to user devices (Check Point, 2022). However, RBI implementations face substantial challenges including high latency due to routing all traffic through cloud infrastructure, significant bandwidth consumption especially with pixel-streaming approaches, and substantial costs that make deployment impractical for many organizations (Palo Alto Networks, n.d.).

Additional RBI challenges include compatibility issues with complex web applications, resolution problems on high-DPI displays, and the requirement for continuous server maintenance (Zenarmor, n.d.).

Furthermore, social engineering attacks and phishing can still succeed in RBI environments because they target user rather than technical vulnerabilities, meaning RBI cannot prevent users from disclosing credentials to convincing fake websites (Zscaler, n.d.).

Problem Statement

Current browser security solutions create a critical trade-off between security effectiveness and operational practicality. RBI provides strong isolation, but latency is high when it comes to complex task, high infrastructure costs thousands of dollars monthly per deployment, and compatibility failures with approximately 15-20% web applications that use dynamic JavaScript and complex rendering (StrongDM, 2025).

Application functionality and productivity losses estimated at 10-15% in high-bandwidth tasks (Menlo Security, n.d.).

Current browsers using traditional containers share the host operating system kernel (Google Chrome Site Isolation / Firefox Sandbox), where a single container can potentially affect all others on the system and vulnerability to kernel-level exploits. (HuggingFace, n.d.).

The Docker container approach, while lightweight, has documented cases of container escape vulnerabilities that allow malware to breach isolation and access the host system directly (Medium, 2024).

Current file monitoring solutions often fail to detect sophisticated malware that employs anti-sandbox detection techniques, with estimates suggesting 560,000 new pieces of malware are detected daily (Cybersecurity Ventures, 2024).

We aim to develop a local browser isolation system using Firecracker-microVMs Because it provides hardware-level isolation, running lightweight virtual machine with its own kernel, offering stronger security and protection.

The system will implement real-time file monitoring with behavioral analysis to detect malicious activities, maintaining detailed logs of all browser and file system interactions. The solution targets minimal resource overhead (under 5MB per isolated session) and native browsing performance comparable to standard browsers, eliminating the latency and cost issues inherent in cloud-based RBI solutions.

Scope and Limitations

We will develop a prototype local browser isolation system targeting Windows operating systems. The system will utilize Firecracker-microVMs for browser isolation, that give us kernel-level separated containers for deployment (AWS, 2020).

We Choose browser Engine as a Chromium engine or Chromium Embedded Framework (CEF), The reason we choose its compatible and customization capabilities.

The file monitoring will implement real-time scanning and post-download.

Real-time scanning through a background process that monitors browser activity / setting and changes in Firecracker file system Also Post-download scanning analysis the files when users download files from the isolated browser. We focus on detecting common threats like malware, ransomware, and suspicious scripts through behavioral analysis techniques such as monitoring unauthorized file system modifications, tracking privilege escalation attempts, and identifying anomalous network communications.

Expected Impact and Stakeholders

Primary stakeholders are

- enterprise IT security teams
- small-to-medium enterprises (SMEs)
- individual users

The expected impacts are

- Reducing browser-based attacks by providing hardware-virtualization-level isolation.
- That prevents malware from reaching the host system even when users visit malicious websites.
- Eliminating the cloud infrastructure dependency and RBI subscription costs
- The file monitoring to provide details about security information about threats.
- Performance improvements over RBI (eliminating latencies).

Chapter 02: Business Case

2.1 Business Need

Organizations face an escalating browser threat landscape with over 752,000 browser-based phishing attacks recorded in 2024, marking a 140% year-over-year increase (Infosecurity Magazine, 2025).

Current security approaches prove inadequate: traditional antivirus tools miss 30% of browser-based evasive threats, while Remote Browser Isolation implementations cost enterprises between \$120,000-\$600,000 annually for 1,000 users when including infrastructure, licensing, and management overhead (Venn, 2025).

Small and medium enterprises report that RBI solutions are "prohibitively expensive" with 73% of SMEs citing cost as the primary barrier to adoption, leaving them reliant on inferior endpoint protection that fails against zero-day exploits (NordLayer, 2025).

The average data breach costs organizations \$4.88 million globally, with browser-based attacks serving as the initial entry point in 41% of all cyber incidents (IBM, as cited in Keepnet, 2025).

2.2 Business Objectives

Objective 1: Cost Reduction

Reduce RBI subscriptions that maximum \$12 per user cost \$120-150 per user annually for cloud RBI. Along with total cost of ownership including hardware, licensing, and operational overhead against equivalent RBI subscriptions for the same user base.

Objective 2: Security Effectiveness

Detect browser-based threats and suspicious file behaviors within the isolated environment. This objective will be validated through testing against standardized malware datasets and simulated attack scenarios.

Target metric is to Block 95% of MITRE ATT&CK framework browser-based techniques with false positive rate under 2%.

Objective 3: Performance Maintenance

Maintain browser Open times and page render performance within of native browser speeds. This ensures user productivity is not sacrificed for security, addressing the primary user complaint about RBI systems.

Objective 4: Operational Efficiency

Reduce security incident investigation time by 60% through automated logging and behavioral analysis that provides actionable forensic data. Security teams will have immediate access to detailed activity logs showing exactly what actions occurred within isolated sessions.

Chapter 03: Project Objectives

Objective 1: Implement Firecracker-based Browser Isolation

Develop a working browser isolation environment using Firecracker-microVMs that achieves complete kernel-level separation from the host operating system.

Malware executed within the isolated browser cannot access host system files, registry keys, or network resources except through defined controlled channels.

Objective 2: Develop Real-time File Monitoring System

Create a dual-layer file monitoring system consisting of a kernel-mode driver or background daemon that continuously monitors the isolated browser's file system activities, and a post-download analyzer that examines downloaded files before allowing access to the host system. The monitoring system will track file creation, modification, deletion, registry changes, and process execution attempts, maintaining a detailed log of all activities.

Objective 3: Build User-Friendly Browser Interface

Develop a lightweight launcher application that enables users to initiate isolated browsing sessions with minimal friction. The interface will provide status indicators for isolation state, file monitoring activity, and threat alerts, while maintaining a familiar browser experience through the integrated Chromium-based engine.

Objective 4: Implement Behavioral Analysis Engine

Create a rule-based behavioral analysis system that identifies malicious patterns such as rapid file encryption (ransomware indicators), unauthorized system file modifications, privilege escalation attempts, and suspicious network communications. The engine will use predefined YARA rules for known malware patterns combined with behavioral heuristics for anomaly detection.

Chapter 04: Method of Approach

System Architecture Components

The system architecture comprises four primary layers working in concert to provide comprehensive browser isolation and threat detection.

Layer 1 implements Firecracker microVM management, handling microVM lifecycle (creation, configuration, execution, termination) through Firecracker's RESTful API. The microVM management module creates isolated environments for each browsing session, configuring minimal guest operating system (Alpine Linux or similar lightweight distribution) with required networking (virtio-net) and storage (virtio-block) interfaces.

Layer 2 consists of the browser engine integration, utilizing Chromium Embedded Framework (CEF) or stripped-down Chromium browser deployed within each microVM instance. Browser configuration removes unnecessary features while maintaining core browsing functionality and modern web standards support.

Layer 3 implements the file system interception and monitoring layer, deploying a background daemon process that monitors all file system activities within the microVM using Linux kernel hooks (inotify or FUSE-based monitoring). This layer tracks file creation, modification, deletion, reads, writes, and metadata changes in real-time.

Layer 4 provides a host-side management and analysis platform, including policy enforcement engine that determines whether file operations should be allowed or blocked, logging system that maintains detailed audit trails, and user interface for session management and threat alerts.

Communication between layers utilizes secure channels: the host communicates with microVMs through Firecracker's API socket and virtio network interfaces, file transfer from isolated environment to host occurs through controlled copy operations with intermediate scanning, and monitoring data streams from microVM to host for analysis and logging. Security boundaries ensure the microVM cannot directly access host filesystem or network resources except through explicitly configured virtio devices, with all interactions logged and subject to policy enforcement.

Data Collection and Security Testing

Malware sample collection for testing will utilize public malware databases including VirusTotal, MalwareBazaar, and the EMBER dataset, focusing on multiple recent threats (2023-2025) to ensure relevance while including historical samples for comprehensive coverage.

Ethical and legal considerations mandate using only publicly available malware samples from legitimate security research repositories, executing samples exclusively in isolated test environments, and obtaining proper authorization for any third-party testing. Data privacy compliance ensures no testing involves actual user data, personal information, or production systems.

Detection and Analysis Techniques

The threat detection engine implements a multi-layered approach combining signature-based, heuristic, and behavioral analysis techniques. YARA rule integration provides pattern-matching capabilities for known malware families, with custom rules developed specifically for browser-based threats including suspicious JavaScript patterns, malicious download behaviors, and common exploit techniques.

Behavioral heuristics monitor for ransomware indicators such as rapid file encryption patterns, high-entropy file writes suggesting encryption operations, mass file renaming or deletion, and attempts to modify shadow copies or backups. Information stealer detection identifies clipboard monitoring, keylogging behaviors, credential file access patterns, and unauthorized data exfiltration attempts through network operations.

System integrity monitoring tracks modifications to critical OS files and registry keys, privilege escalation attempts through process token manipulation or service creation, and injection attacks targeting legitimate processes. Network behavioral analysis examines outbound connection attempts to known command-and-control infrastructure, unusual DNS queries suggesting malware beaconing, and data exfiltration through covert channels.

The analysis engine maintains a configurable rule set enabling administrators to customize detection sensitivity, define allowed behaviors for specific applications, and implement organization-specific security policies. Machine learning integration remains scope-limited but the architecture supports future enhancement with trained models for anomaly detection and unknown threat classification.

Performance Optimization Strategy

Performance targets drive architectural decisions throughout implementation. Firecracker initialization optimization includes pre-built microVM templates with browser engine pre-installed, lazy loading of browser components to accelerate initial launch, and memory-mapped file systems reducing disk I/O overhead.

Resource management implements intelligent session pooling where recently closed microVMs remain in standby state for rapid reuse, configurable memory limits preventing resource exhaustion from multiple simultaneous sessions, and CPU pinning strategies to minimize context switching overhead.

File monitoring efficiency utilizes kernel-level hooks rather than userspace polling to reduce CPU consumption, event filtering to process only security-relevant operations rather than every filesystem interaction, and asynchronous logging to prevent I/O operations from blocking main execution.

Browser performance tuning disables unnecessary Chromium features (developer tools, extensions infrastructure, sync capabilities) that add overhead without security benefit, enables hardware acceleration where available in virtualized environment, and implements connection keep-alive for frequently accessed sites.

Development Tools and Technologies

Core implementation languages include

- C/C++ for performance-critical components such as file system monitoring hooks and microVM management,
- Python for orchestration, logging, and analysis systems where rapid development is prioritized over raw performance,
- Planning to use Rust for security-critical modules where memory safety is paramount.

Development frameworks leverage Firecracker SDK for microVM lifecycle management, Chromium Embedded Framework (CEF) for browser integration providing programmatic control over browser instances, and Linux kernel APIs (inotify, fanotify, eBPF) for file system monitoring.

Security testing tools encompass static analysis using Clang Static Analyzer and Coverity Scan for code vulnerability identification, dynamic analysis with Valgrind for memory error detection and AddressSanitizer for runtime error catching, and fuzzing techniques using AFL or LibFuzzer to discover edge cases and potential exploits.

Malware analysis tools include Cuckoo Sandbox for automated dynamic malware analysis, VirusTotal API for malware sample validation and multi-engine scanning, and YARA for rule-based malware identification. Logging and monitoring infrastructure utilizes structured JSON logging for machine-readable audit trails, ELK stack (Elasticsearch, Logstash, Kibana) for log aggregation and analysis in development environment, and Prometheus/Grafana for performance metrics visualization during testing.

Chapter 05: Conceptual Diagram

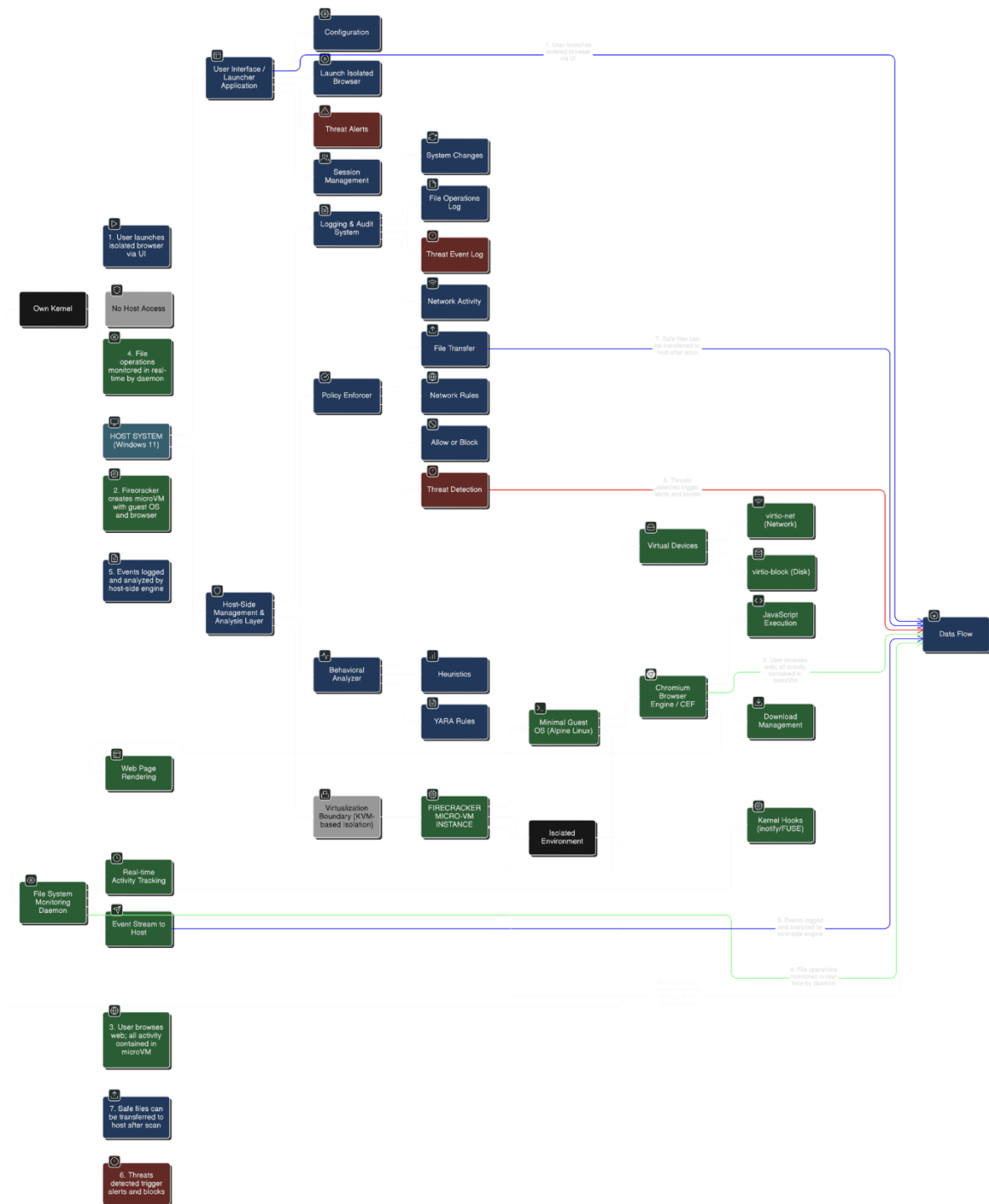


Diagram Explanation:

The architecture implements defense-in-depth through multiple isolation boundaries. The Firecracker microVM provides hardware-virtualization-level isolation ensuring the browser and any malware cannot access the host system's kernel, filesystem, or memory.

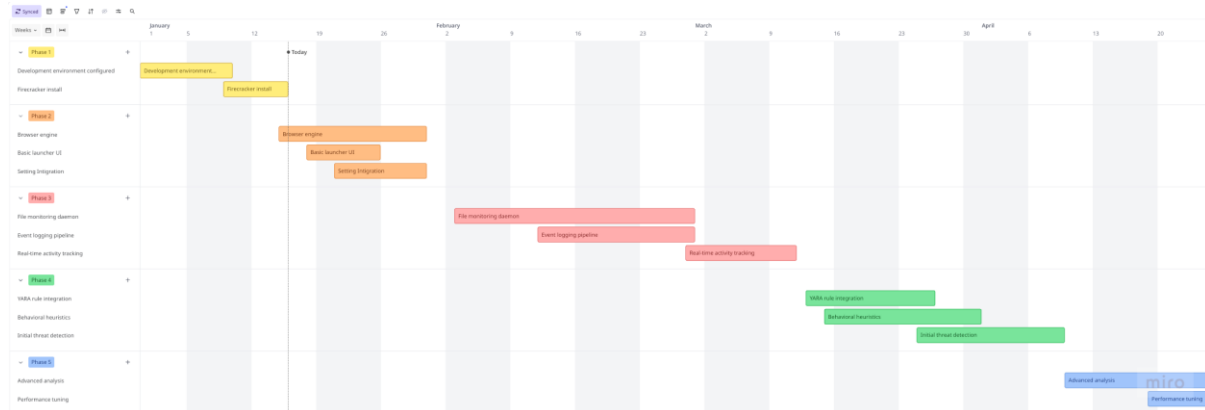
Within the microVM, the file monitoring daemon operates at the guest kernel level, capturing all file operations before they can cause harm.

The host-side analysis layer examines event streams from multiple microVM instances simultaneously, applying behavioral analysis and policy enforcement across all active browsing sessions.

This multi-layer approach ensures that even if one security layer is bypassed, additional layers provide backup protection. Communication between layers uses defined, secure channels (Firecracker API, virtio interfaces) that are logged and monitored, providing complete audit trails for security investigations.

Chapter 06: Initial Project Plan

Timeline and Milestones



Resource Requirements

Hardware Resources:

- Development Workstation: Intel Core i7 or AMD Ryzen 7, 16GB RAM, 512GB NVMe SSD, Windows 11 Pro with Hyper-V or bare-metal Linux with KVM
- Testing Hardware: Secondary machine or cloud compute instance (AWS i3.metal or equivalent) for performance benchmarking and security testing
- Network Infrastructure: Isolated network segment for malware testing, preventing accidental spread

Software Resources:

- Development Tools: Visual Studio Code, CMake, GCC/Clang compilers, Python 3.10+, Git version control
- Firecracker: Latest stable release from GitHub repository, requiring Linux kernel 4.14+ with KVM support
- Browser Engine: Chromium source code or CEF binary distribution
- Analysis Tools: YARA, Cuckoo Sandbox, VirusTotal API access, ELK stack for logging
- Operating Systems: Windows 11 Pro (host), Alpine Linux (guest OS for microVMs)

Knowledge Resources:

- Technical documentation: Firecracker API reference, Linux kernel documentation, Chromium architecture guides
- Security resources: MITRE ATT&CK framework, CVE database, malware analysis tutorials
- Academic papers: Browser isolation research, virtualization security, behavioral malware detection

Chapter 07: Risk Analysis

Risk 1: Firecracker Compatibility Issues

Firecracker requires specific hardware and kernel support that may not be available on all development or deployment systems. The technology demands KVM-capable processors with virtualization extensions enabled and Linux kernel 4.14 or newer. On Windows development environments, this necessitates either WSL2 with nested virtualization or dual-boot configuration.

Mitigation Strategy: Conduct early compatibility testing on target hardware before committing to Firecracker. Establish cloud development environment (AWS i3.metal instances) as fallback option if local hardware proves incompatible. Document minimum system requirements clearly for future deployments. Consider alternative virtualization technologies (lightweight VMs using VirtualBox or VMware) as last resort if Firecracker proves unworkable, accepting performance tradeoffs.

Risk 2: Browser Integration Complexity

Integrating Chromium/CEF within Firecracker microVMs presents multiple technical challenges including ensuring browser dependencies are available in minimal guest OS, managing GPU acceleration in virtualized environment, and handling browser updates and security patches. Browser engines have complex dependency trees that lightweight Alpine Linux may not satisfy completely.

Mitigation Strategy: Begin with minimal browser functionality, progressively adding features as integration matures. Use pre-compiled CEF binaries rather than building from source to reduce complexity. Implement automated build scripts that package browser with all dependencies verified working. If CEF integration proves too complex, fallback to lightweight alternatives like QtWebEngine or even text-based browsers (w3m, lynx) for proof-of-concept, deferring full-featured browser to future iterations.

Risk 3: Performance Targets Unmet

The system may fail to achieve target performance metrics (<150ms initialization, <10% overhead) due to virtualization overhead, file monitoring CPU consumption, or inefficient communication between microVM and host. Performance shortfalls could render the system impractical for real-world usage despite strong security properties.

Mitigation Strategy: Implement performance monitoring throughout development, catching issues early. Profile code regularly using tools like Valgrind, perf, and flamegraphs to identify bottlenecks. Optimize critical paths first (microVM initialization, page rendering). Consider trading features for performance if necessary—for example, reducing logging granularity or simplifying detection heuristics to meet performance targets. Establish minimum acceptable performance thresholds and be prepared to adjust scope if fundamental limitations are discovered.

Risk 4: Detection Accuracy Issues

The behavioral analysis engine may produce too many false positives (legitimate software flagged as threats) or false negatives (actual malware missed), reducing system usefulness. Balancing sensitivity versus specificity presents significant challenge, particularly for unknown malware employing novel techniques.

Mitigation Strategy: Build comprehensive test dataset spanning benign software and diverse malware families. Implement configurable sensitivity levels allowing administrators to tune detection aggressiveness. Maintain detailed logs of all detections with full context enabling manual review and rule refinement. Establish feedback mechanism for false positive reporting and continuously improve detection rules based on real-world results. Start with conservative rules (high confidence threshold) and progressively add more aggressive detection as false positive rate is validated.

Risk 5: Limited Malware Sample Access

Obtaining diverse, high-quality malware samples for testing may prove difficult due to access restrictions, legal concerns, or limited availability of recent samples in public repositories. Insufficient test data compromises validation of detection capabilities.

Mitigation Strategy: Utilize multiple public malware sources including VirusTotal, MalwareBazaar, Hybrid Analysis, and academic datasets like EMBER. Focus on well-documented malware families with published behavioral characteristics. Consider creating synthetic malware samples that exhibit specific behaviors without actual malicious functionality for testing. Collaborate with university security lab or industry contacts who may provide additional samples. Document testing methodology thoroughly such that additional validation can be conducted post-project as new samples become available.

References

Amazon Science (2021). How AWS's Firecracker virtual machines work. Available at: <https://www.amazon.science/blog/how-awss-firecracker-virtual-machines-work> [Accessed 14 January 2026].

AWS (2020). Announcing the Firecracker Open Source Technology: Secure and Fast microVM for Serverless Computing. AWS Open Source Blog. Available at: <https://aws.amazon.com/blogs/opensource/firecracker-open-source-secure-fast-microvm-serverless/> [Accessed 14 January 2026].

Check Point (2022). What is Remote Browser Isolation (RBI)? Check Point Software. Available at: <https://www.checkpoint.com/cyber-hub/threat-prevention/what-is-remote-browser-isolation-rbi/> [Accessed 14 January 2026].

Cloudflare (n.d.). What is browser isolation? | Remote browser isolation. Cloudflare Learning Center. Available at: <https://www.cloudflare.com/learning/access-management/what-is-browser-isolation/> [Accessed 14 January 2026].

CrowdStrike (2025). Malware Analysis: Steps & Examples. CrowdStrike Cybersecurity 101. Available at: <https://www.crowdstrike.com/en-us/cybersecurity-101/malware/malware-analysis/> [Accessed 14 January 2026].

Cybersecurity Ventures (2024). Web Browsers Are Doorways To Cyberattacks. Available at: <https://cybersecurityventures.com/web-browsers-are-doorways-to-cyberattacks/> [Accessed 14 January 2026].

Firecracker GitHub (n.d.). Secure and fast microVMs for serverless computing. GitHub Repository. Available at: <https://github.com/firecracker-microvm/firecracker> [Accessed 14 January 2026].

Firecracker (n.d.). Firecracker. Official Website. Available at: <https://firecracker-microvm.github.io/> [Accessed 14 January 2026].

Google Cloud Blog (2024). (QR) Coding My Way Out of Here: C2 in Browser Isolation Environments. Google Cloud Threat Intelligence. Available at: <https://cloud.google.com/blog/topics/threat-intelligence/c2-browser-isolation-environments> [Accessed 14 January 2026].

HuggingFace (n.d.). Firecracker vs Docker: The Technical Boundary Between MicroVMs and Containers. HuggingFace Blog. Available at: <https://huggingface.co/blog/agentbox-master/firecracker-vs-docker-tech-boundary> [Accessed 14 January 2026].

Infosecurity Magazine (2025). 752,000 Browser Phishing Attacks Mark 140% Increase YoY. Available at: <https://www.infosecurity-magazine.com/news/752000-browser-phishing-attacks/> [Accessed 14 January 2026].

Keepnet (2025). 300 Cyber Security Statistics, Facts, Figures (Nov 2025). Keepnet Labs Blog. Available at: <https://keepnetlabs.com/blog/171-cyber-security-statistics-2024-s-updated-trends-and-data> [Accessed 14 January 2026].

Medium (2024). Understanding Firecracker MicroVMs: The Next Evolution in Virtualization. Medium. Available at: <https://medium.com/@meziounir/understanding-firecracker-microvms-the-next-evolution-in-virtualization-cb9eb8bbeede> [Accessed 14 January 2026].

Menlo Security (2024). Browser-Based Phishing Attacks Increased 198% in 2023 as Threat Actors Grow More Evasive. Menlo Security Press Release. Available at: <https://www.menlosecurity.com/press-releases/browser-based-phishing-attacks-increased-198-in-2023-as-threat-actors-grow-more-evasive-menlo-security-research-finds> [Accessed 14 January 2026].

Menlo Security (n.d.). Remote Browser Isolation. Menlo Security Product Documentation. Available at: <https://www.menlosecurity.com/product/remote-browser-isolation> [Accessed 14 January 2026].

NordLayer (2025). What Is Remote Browser Isolation (RBI) and How Does It Work? NordLayer Learn. Available at: <https://nordlayer.com/learn/browser-security/what-is-browser-isolation/> [Accessed 14 January 2026].

OpenMetal (2025). MicroVMs: Scaling Out Over Scaling Up in Modern Cloud Architectures. OpenMetal IaaS Blog. Available at: <https://openmetal.io/resources/blog/microvms-scaling-out-over-scaling-up/> [Accessed 14 January 2026].

Palo Alto Networks (n.d.). What Is Remote Browser Isolation (RBI)? Palo Alto Networks Cyberpedia. Available at: <https://www.paloaltonetworks.com/cyberpedia/what-is-remote-browser-isolation> [Accessed 14 January 2026].

Proofpoint (2024). What Is Web Browser Isolation? - Remote Browser Isolation. Proofpoint Threat Reference. Available at: <https://www.proofpoint.com/us/threat-reference/browser-isolation> [Accessed 14 January 2026].

ScienceDirect (2021). A study on malicious software behaviour analysis and detection techniques: Taxonomy, current trends and challenges. Future Generation Computer Systems. Available at: <https://www.sciencedirect.com/science/article/abs/pii/S0167739X21004751> [Accessed 14 January 2026].

Security Magazine (2024). Browser-based phishing attacks increased 198% in H2 2023. Available at: <https://www.securitymagazine.com/articles/100343-browser-based-phishing-attacks-increased-198-in-h2-2023> [Accessed 14 January 2026].

StrongDM (2025). What is Remote Browser Isolation? RBI Explained. StrongDM Blog. Available at: <https://www.strongdm.com/blog/remote-browser-isolation> [Accessed 14 January 2026].

Terranova Security (2024). 130 Cyber Security Statistics: 2024 Trends and Data. Terranova Security Blog. Available at: <https://www.terrnovasecurity.com/blog/cyber-security-statistics> [Accessed 14 January 2026].

Trend Micro (n.d.). Faster and More Accurate Malware Detection Through Predictive Machine Learning. Trend Micro Security News. Available at: <https://www.trendmicro.com/vinfo/us/security/news/security-technology/faster-and-more-accurate-malware-detection-through-predictive-machine-learning-correlating-static-and-behavioral-features> [Accessed 14 January 2026].

Varonis (2025). Top 11 Malware Analysis Tools and Their Features. Varonis Blog. Available at: <https://www.varonis.com/blog/malware-analysis-tools> [Accessed 14 January 2026].

Venn (2025). Remote Browser Isolation: Challenges, Alternatives, and Best Practices. Venn Browser Security. Available at: <https://www.venn.com/learn/browser-security/remote-browser-isolation/> [Accessed 14 January 2026].

VMRay (2025). Malware Detection Techniques: The Complete Guide. VMRay Blog. Available at: <https://www.vmrays.com/malware-detection-techniques-the-complete-guide/> [Accessed 14 January 2026].

Wikipedia (2025). Browser isolation. Available at: https://en.wikipedia.org/wiki/Browser_isolation [Accessed 14 January 2026].

Zenarmor (n.d.). What is Remote Browser Isolation (RBI)? Zenarmor Documentation. Available at: <https://www.zenarmor.com/docs/network-security-tutorials/what-is-remote-browser-isolation-rbi> [Accessed 14 January 2026].

Zscaler (n.d.). What Is Remote Browser Isolation? Need & Benefits. Zscaler Resources. Available at: <https://www.zscaler.com/resources/security-terms-glossary/what-is-remote-browser-isolation> [Accessed 14 January 2026].